

PRACTICAL-8

Task 1: Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;
    int *malloc_ptr;
    int *calloc_ptr;
    int *realloc_ptr;

    printf("Dynamic Memory Allocation Demonstration\n");
    printf("-----\n");

    /* malloc() */
    malloc_ptr = (int *)malloc(5 * sizeof(int));

    if (malloc_ptr == NULL) {
        printf("malloc() allocation failed.\n");
        return 1;
    }

    printf("\nmalloc(): Allocated memory for 5 integers.\n");

    for (i = 0; i < 5; i++) {
        malloc_ptr[i] = (i + 1) * 10;
    }

    printf("Values using malloc(): ");
    for (i = 0; i < 5; i++) {
        printf("%d ", malloc_ptr[i]);
    }
    printf("\n");

    /* calloc() */
    calloc_ptr = (int *)calloc(5, sizeof(int));

    if (calloc_ptr == NULL) {
        printf("calloc() allocation failed.\n");
        free(malloc_ptr);
        return 1;
    }

    printf("\ncalloc(): Allocated memory for 5 integers.\n");
```

```

printf("Initial calloc() values: ");
for (i = 0; i < 5; i++) {
    printf("%d ", calloc_ptr[i]);
}
printf("\n");

/* realloc() */
realloc_ptr = (int *)malloc(3 * sizeof(int));

if (realloc_ptr == NULL) {
    printf("Initial realloc() allocation failed.\n");
    free(malloc_ptr);
    free(calloc_ptr);
    return 1;
}

for (i = 0; i < 3; i++) {
    realloc_ptr[i] = (i + 1) * 100;
}

printf("\nrealloc(): Initially allocated memory for 3 integers.\n");

realloc_ptr = (int *)realloc(realloc_ptr, 6 * sizeof(int));

if (realloc_ptr == NULL) {
    printf("realloc() failed.\n");
    free(malloc_ptr);
    free(calloc_ptr);
    return 1;
}

for (i = 3; i < 6; i++) {
    realloc_ptr[i] = (i + 1) * 100;
}

printf("realloc(): Expanded memory from 3 to 6 integers.\n");

printf("Values after realloc(): ");
for (i = 0; i < 6; i++) {
    printf("%d ", realloc_ptr[i]);
}

```

Help
Write Out
Where Is
Cut
Execute

```

    free(malloc_ptr);
    free(calloc_ptr);
    return 1;
}

for (i = 3; i < 6; i++) {
    realloc_ptr[i] = (i + 1) * 100;
}

printf("realloc(): Expanded memory from 3 to 6 integers.\n");

printf("Values after realloc(): ");
for (i = 0; i < 6; i++) {
    printf("%d ", realloc_ptr[i]);
}
printf("\n");

/* free() */
free(malloc_ptr);
printf("\nfree(): malloc() memory released.\n");

free(calloc_ptr);
printf("free(): calloc() memory released.\n");

free(realloc_ptr);
printf("free(): realloc() memory released.\n");

printf("\nAll dynamically allocated memory has been released.\n");

return 0;
}

```

```

ruhi0404@Ruhitha:~/Operating_Systems$ nano prac8task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc -Wall -Wextra prac8task1.c -o prac8task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./prac8task1
Dynamic Memory Allocation Demonstration
-----

malloc(): Allocated memory for 5 integers.
Values using malloc(): 10 20 30 40 50

calloc(): Allocated memory for 5 integers.
Initial calloc() values: 0 0 0 0 0

realloc(): Initially allocated memory for 3 integers.
realloc(): Expanded memory from 3 to 6 integers.
Values after realloc(): 100 200 300 400 500 600

free(): malloc() memory released.
free(): calloc() memory released.
free(): realloc() memory released.

All dynamically allocated memory has been released.

```

Observation:

The program successfully demonstrated dynamic memory management using `malloc()`, `calloc()`, `realloc()`, and `free()`. `malloc()` allocated memory for five integers, while `calloc()` allocated five integers with their initial values set to zero. `realloc()` was used to increase an existing allocation from three integers to six integers. Finally, `free()` released all dynamically allocated memory.

Task-2: Develop a program that demonstrates Copy-on-Write behavior after a `fork()`. Analyze memory usage before and after modifying data in the child process.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define SIZE 1000000

int main() {
    int *data;
    pid_t pid;

    data = (int *)malloc(SIZE * sizeof(int));

    if (data == NULL) {
        perror("malloc");
        return 1;
    }

    /* Initialize memory */
    for (int i = 0; i < SIZE; i++) {
        data[i] = i;
    }

    printf("Copy-on-Write Demonstration\n");
    printf("-----\n");

    printf("Parent PID: %d\n", getpid());
    printf("Allocated memory: %zu bytes\n", SIZE * sizeof(int));

    printf("\nMemory is initialized in the parent.\n");
    printf("First value: %d\n", data[0]);
    printf("Last value: %d\n", data[SIZE - 1]);

    fflush(stdout);

    pid = fork();

    if (pid < 0) {
        perror("fork");
        free(data);
        return 1;
    }

```

^G Help
 ^X Exit

^O Write Out
 ^R Read File

^W Where Is
 ^\ Replace

^K Cut
 ^U Paste

^T Execute
 ^J Justify

```
    printf("data[SIZE-1] = %d\n", data[SIZE - 1]);

    printf("\nChild is modifying the shared memory...\n");

    /* Modify memory */
    for (int i = 0; i < SIZE; i++) {
        data[i] = data[i] + 1000;
    }

    printf("\nAfter modification in child:\n");
    printf("data[0] = %d\n", data[0]);
    printf("data[SIZE-1] = %d\n", data[SIZE - 1]);

    printf("\nCopy-on-Write occurred when the child modified the pages.\n");

    free(data);
    exit(0);
}
else {
    /* Parent process */
    printf("\nParent created child process.\n");
    printf("Child PID: %d\n", pid);

    printf("\nParent data before child modification:\n");
    printf("data[0] = %d\n", data[0]);
    printf("data[SIZE-1] = %d\n", data[SIZE - 1]);

    wait(NULL);

    printf("\nParent data after child modification:\n");
    printf("data[0] = %d\n", data[0]);
    printf("data[SIZE-1] = %d\n", data[SIZE - 1]);

    printf("\nParent values remain unchanged.\n");
    printf("This demonstrates Copy-on-Write behavior.\n");

    free(data);
}

return 0;
}
```

```

[1]~ Stopped                  sudo apt update
ruhi0404@Ruhitha:~/Operating_Systems$ nano prac8task2.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc -Wall -Wextra prac8task2.c -o prac8task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./prac8task2
Copy-on-Write Demonstration
-----
Parent PID: 1531
Allocated memory: 4000000 bytes

Memory is initialized in the parent.
First value: 0
Last value: 999999

Parent created child process.
Child PID: 1532

Parent data before child modification:
data[0] = 0
data[SIZE-1] = 999999

Child process started.
Child PID: 1532
Parent PID: 1531

Before modification:
data[0] = 0
data[SIZE-1] = 999999

Child is modifying the shared memory...

After modification in child:
data[0] = 1000
data[SIZE-1] = 1000999

Copy-on-Write occurred when the child modified the pages.

Parent data after child modification:
data[0] = 0
data[SIZE-1] = 999999

Parent values remain unchanged.
This demonstrates Copy-on-Write behavior.
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation:

The program successfully demonstrated Copy-on-Write behavior using `fork()`. A large dynamically allocated array was initialized in the parent process before creating the child. After `fork()`, the parent and child initially had access to the same logical memory contents.

When the child modified the array, the operating system created private copies of the affected memory pages instead of immediately copying the entire address space at the time of `fork()`. The modified values were visible only in the child, while the parent retained the original values.

Memory usage can be examined using `/proc/<PID>/status` and `/proc/<PID>/smaps_rollup`. These interfaces help distinguish virtual memory from resident, shared, and private memory and demonstrate the memory effects associated with Copy-on-Write.